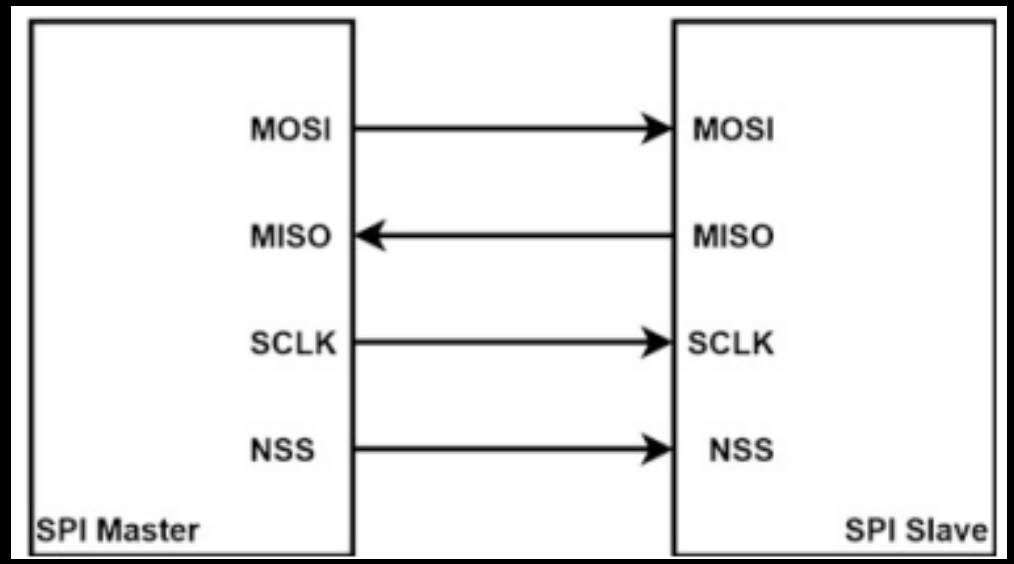
ee2004

week13

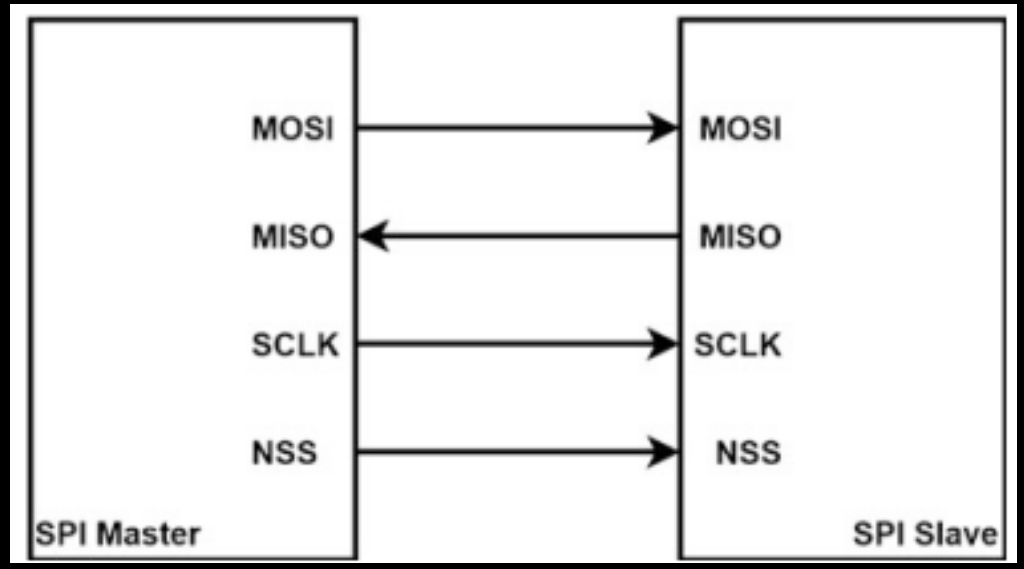
Serial Peripheral Interface
SPI

- SPI is generally used for high-speed serial data transfer. Hence, it is suitable for transferring data with large size.
- We can summarize SPI properties based on the definitions in Sect. 8.1 as follows.
- SPI has an 8 or 16 bit frame consisting only data. Hence, no start, stop, or parity bits are used in the frame.
- SPI uses NRZ data encoding.
- SPI can work in half or full duplex mode. Moreover, if we do not receive data from the slave in the full duplex mode (we do not use one of the MISO or MOSI pins, to be explained next), actually it works in simplex mode.
- SPI can work in either master or slave mode based on the application. The master device generates the clock signal. More than one slave device can be connected to the communication line. In this setting, the SPI master chip select (CS) pins should be used to select which slave to be communicated to. In this setup, one dedicated CS pin should be used for each slave device. If one slave device is used in data transfer, its CS pin can be set to ground or *VDD* based on its being active low or high, respectively.
- There is no limit for data transmission speed in SPI. As in UART, baud rate directly depends on the microcontroller and its clock used in operation.

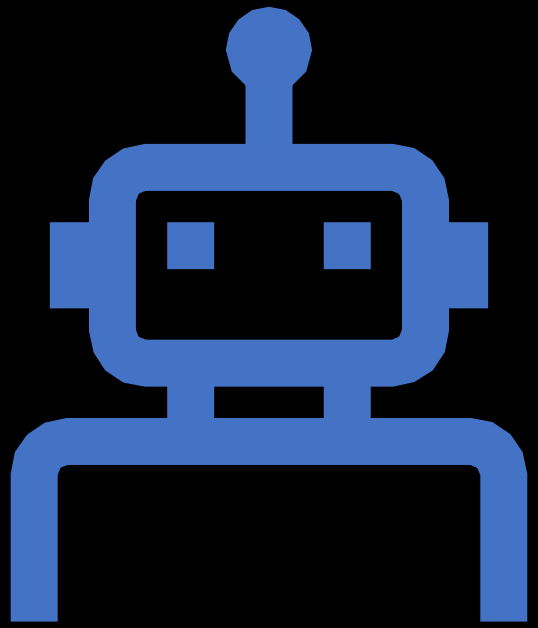
- In order to explain working principles of SPI, we should take a look at the connection diagram between the transmitter and receiver given in Fig. 8.10.
- As can be seen in this figure, there is one master and one slave device in operation. There are four pins in operation as master in/slave out (MISO), master out/slave in (MOSI), serial clock (SCLK), and not slave select (NSS).
- The same named pins are connected. However, they work in reverse order as follows.
- When the device is in master mode, MISO works for data input, MOSI for data output, SCLK as clock output, and NSS as slave select output (in active low mode).
- When the device is in slave mode, MISO works for data output, MOSI for data input, SCLK as clock input, and NSS as slave select input (in active low mode).



- Data transfer between the master and slave devices occurs in serial mode between MOSI and MISO pins in Fig. 8.10.
- The most significant bit is sent first in data transfer.
- Moreover, communication is initiated by the master device.
- When the master device transmits data to a slave device via the MOSI pin, the slave device responds via the MISO pin.
- This implies full duplex communication with both data out and data in, synchronized with the same clock signal (which is provided by the master device via the SCK pin).



SPI Modules in the STM32F4 Microcontroller



- The STM32F4 microcontroller has six SPI modules as SPI1, SPI2, SPI3, SPI4, SPI5, and SPI6.
- Each module can work either in master or slave mode.
- They can also work in half or full duplex modes.
- SPI modules use the peripheral clock as the clock signal.
- The master device has its own clock prescaler so that the clock signal can be divided by 2, 4, 8, 16, 32, 64, 128, or 256. The resulting clock signal will decide on the baud rate.
- To be more specific, SPI1, SPI4, SPI5, and SPI6 modules use the APB2 peripheral clock (PCLK2). Hence, the maximum baud rate can be 45 Mbits/s.
- The SPI2 and SPI3 modules use the APB1 peripheral clock (PCLK1). This may result in the maximum baud rate to 22.5 Mbits/s.

- As can be seen in this figure, there is a shift register in the module. It has an essential role in data transfer.
- The communication and CRC controller blocks are responsible for reading and writing data coming from software to FIFO buffer modules. Moreover, they are responsible for interrupt generation.
- If the SPI module is selected as master, the NSS logic module becomes responsible for setting the output and selecting the slave device.
- If the SPI module is selected as slave, then the NSS logic module activates the read/write operation. The NSS logic module also disables the shift register if it is not selected. This is done to eliminate data confusion if more than one slave device is present in operation.
- The baud rate generator module is responsible for generating the SPI clock signal when the module works as master.
- The register block keeps the modification bits responsible for setting up all properties of the SPI module.

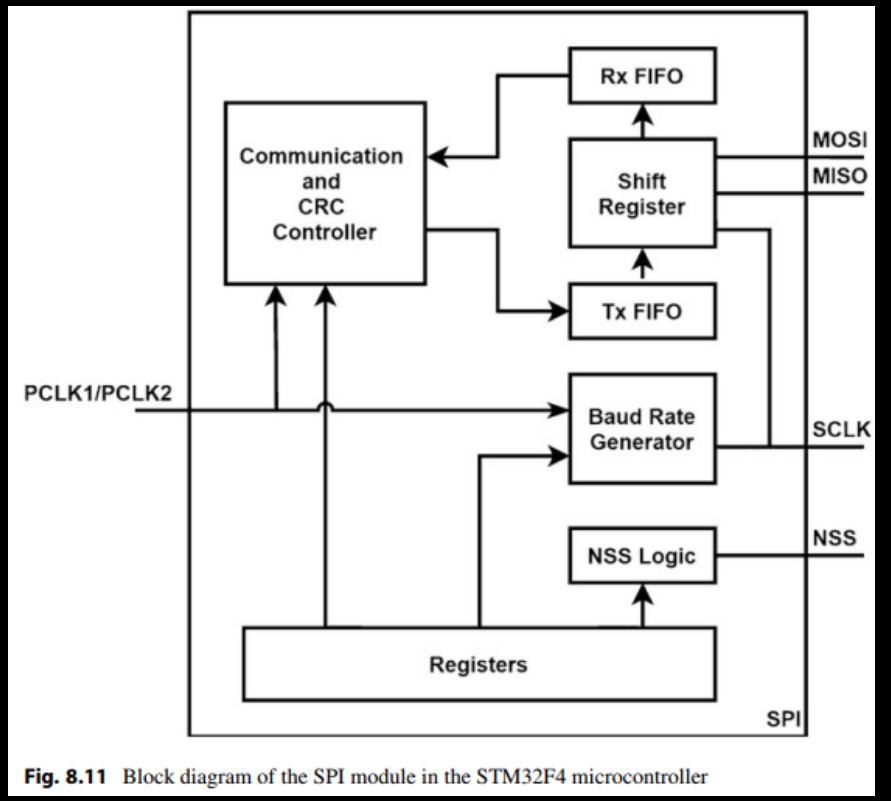
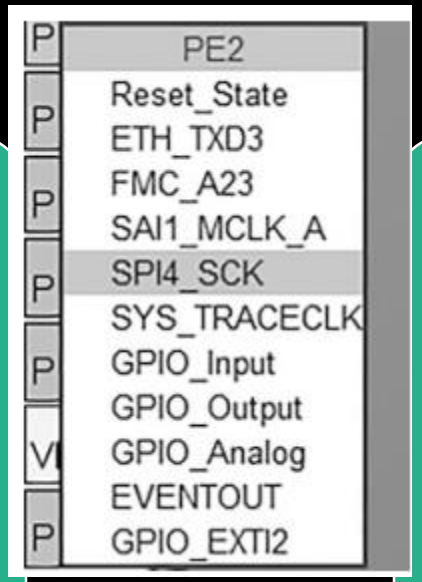


Fig. 8.11 Block diagram of the SPI module in the STM32F4 microcontroller



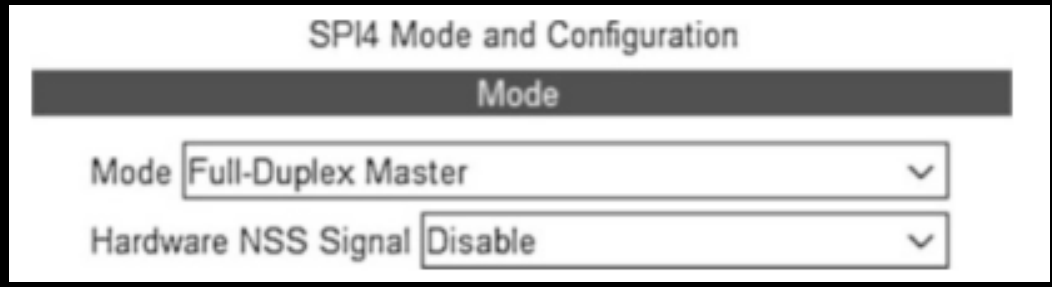
- We can explain working principles of the SPI module as follows.
- Shift registers in the master and slave devices feed 1 bit to their output at each clock pulse generated by the master device.
- Since the master output pin is connected to the slave input pin, the bit fed from the master device shift register is received by the shift register in the slave device.
- Likewise, the master input is connected to the slave output. Hence, the bit fed from the shift register of the slave device is fed to the shift register in the master device.
- As the master device generates eight clock pulses, 1 data byte is transferred between the master and slave devices.
- As the transfer operation ends, the master or slave device moves data in the shift register to the Rx FIFO buffer. Hence, data read cycle is complete.
- If the master or slave device will transfer new data, it is first written to the Tx FIFO buffer. As the data in the shift register is moved to the Rx FIFO buffer, this data is stored in the shift register.

SPI modules in the STM32F4 microcontroller support hardware cyclic redundancy check (CRC). This allows using an SD card via SPI. As a side note, the SD card sends 2 bytes of CRC information in its data packets. This should be decoded first. Since the SPI module in the STM32F4 microcontroller handles this operation, the SD card can be used by it.

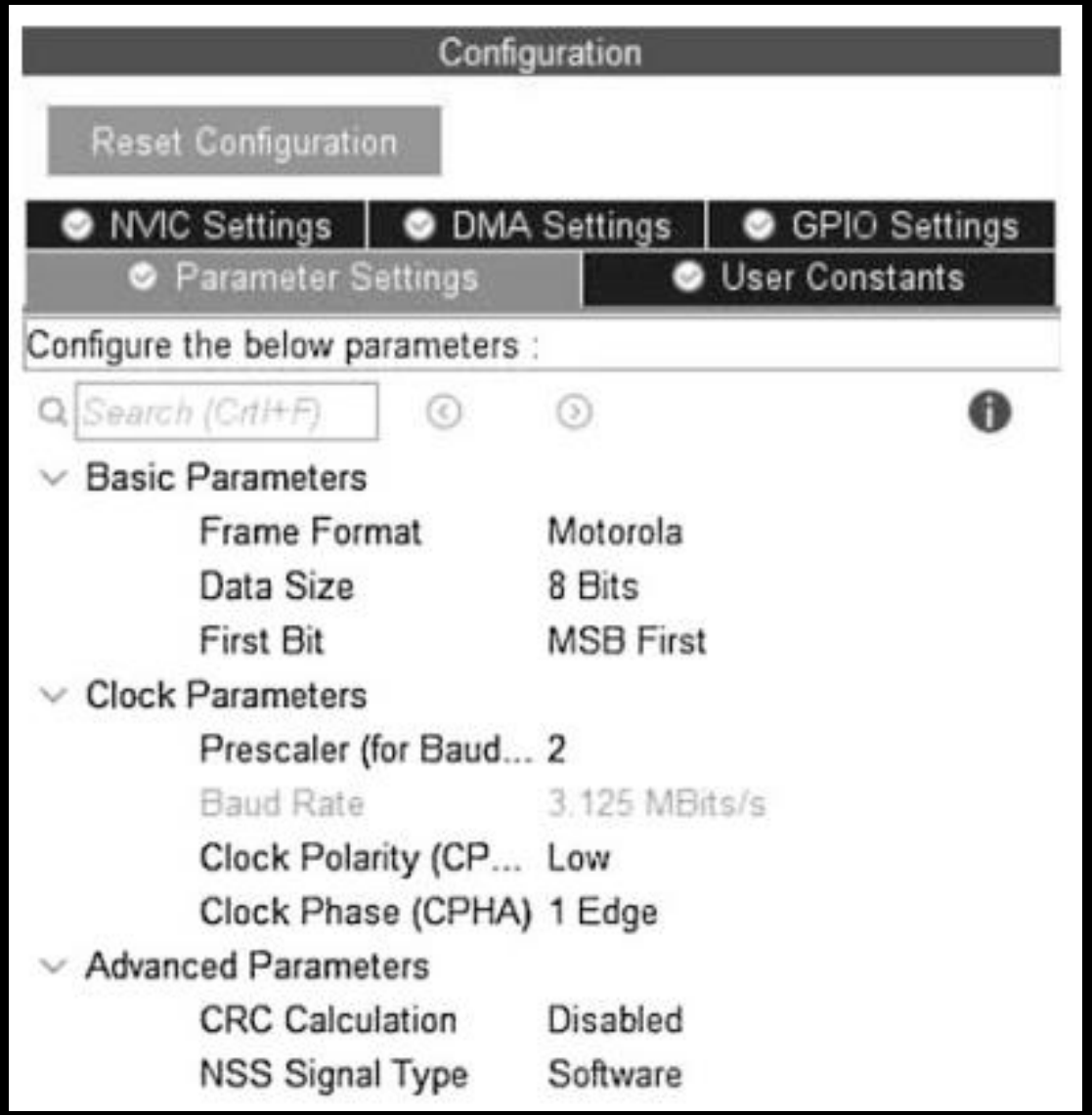


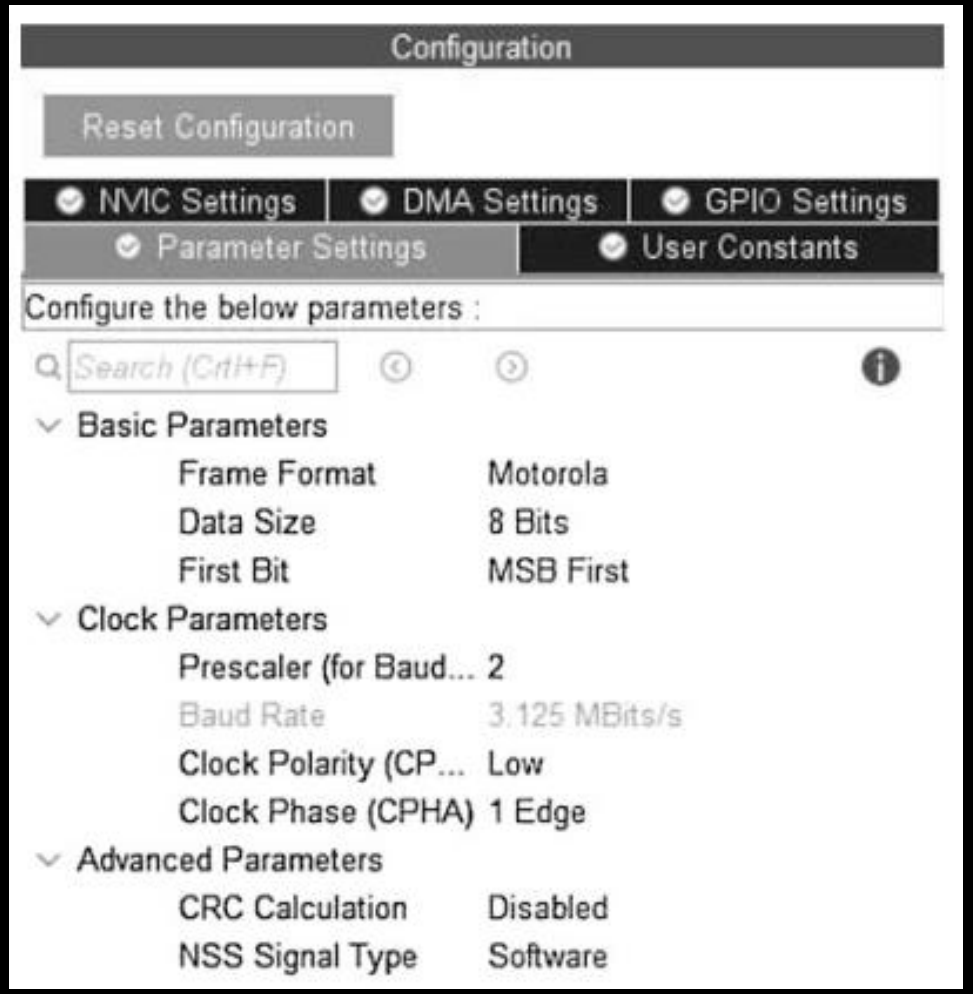
In order to use the SPI module, MISO, MOSI, and SCK (and NSS if it will be used) pins should be configured first. To do so, click on the GPIO pin with SPI capability in the STM32F4 microcontroller in the STM32CubeMX interface.

- Once the SPI pins are selected and configured, click the set SPI module under the “Connectivity” section from the left menu of STM32CubeMX. Then, mode configuration can be done as shown in Fig. 8.13. Here, the mode can be one of “Full-Duplex Master,” “Full-Duplex Slave,” “Half-Duplex Master,” “Half-Duplex Slave,” “Receive-Only Master,” “Receive-Only Slave,” “Transmit-Only Master,” or “Transmit-Only Slave.” Also, the hardware NSS signal can be enabled by clicking its checkbox.



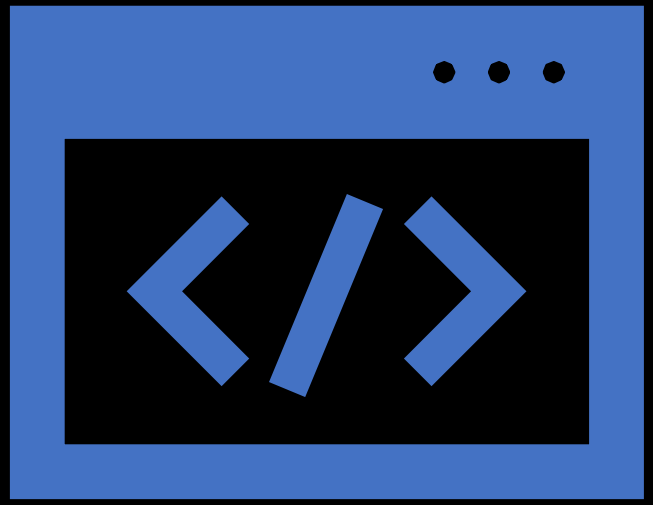
- After configuring the SPI mode, its parameters can be set from the opened “Parameter Settings” tab in the “Configuration” menu as in Fig. 8.14.
- Here, frame format can be selected as one of “Motorola” or “TI,” data size can be selected as one of “8-bit” or “16-bit,” and the first bit can be selected as “MSB First” or “LSB First” under “Basic Parameters.”
- The “TI” format can be only selected if hardware NSS signal is enabled.
- The clock prescaler for baud rate, clock polarity, and phase can be configured under “Clock Parameters.” The clock prescaler is available only for the master device.
- CRC calculation can be enabled under “Advanced Parameters.”





- Similar to UART, SPI can operate in two operation modes as blocking (polling) or non-blocking (interrupt).
- If interrupt generation is not required, then this step can be skipped. Otherwise, the SPI interrupt should be enabled from the NVIC tab in the configuration menu as in Fig. 8.14.
- The interrupt priority can be changed under the “System Core - NVIC” section. To note here, it is suggested to lower the SPI interrupt priority since HAL functions use the SysTick interrupt to create precise 1 ms ticks for internal operations and the HAL_Delay function.
- The SysTick interrupt has the highest priority among peripheral units. Hence, assigning a lower priority to the SPI interrupt will not affect it. Otherwise, precision of the HAL_Delay function may degrade.

Setup via C++ Language



- We can benefit from Mbed and available functions there to adjust the SPI module properties in C++ language.
- To do so, we should first adjust MOSI, MISO, SCK, and NSS pins. This can be done by the classes SPI and SPISlave for master and slave modes, respectively.
- We should supply MOSI, MISO, SCK, and NSS pin names to the class SPI or SPISlave.
- Parameters for both classes, such as frequency, data size, clock polarity, and clock phase, can be configured after initialization.
- Moreover, the operation mode can be selected as blocking or non-blocking.
- A callback function can be called whenever an interrupt occurs in non-blocking mode.

- The first example, with the C code given in Listing 8.10, aims to familiarize the reader with the setup and usage of the SPI module.
- Here, SPI4 and SPI5 modules will be configured as master and slave, respectively.
- There are two counter values in the code.
- The first one will be sent to the slave from master. Upon receiving this data by the slave, the green LED connected to pin PG13 of the STM32F4 board will toggle if mod 3 of the received counter value is equal to zero.
- Afterward, the second counter will be sent to master from slave. Upon receiving this data by the master, the red LED connected to pin PG14 of the STM32F4 board will toggle if mod 5 of the received counter value is equal to zero.
- This operation will repeat every 2 s.

Listing 8.10 Usage of the SPI module in blocking operation mode, the C code

```
/* USER CODE BEGIN PV */
uint8_t masterTxData = 0;
uint8_t slaveTxData = 0;
uint8_t masterRxData;
uint8_t slaveRxData;
/* USER CODE END PV */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
masterTxData++;
slaveTxData++;

HAL_SPI_TransmitReceive(&hspi4, &masterTxData, &masterRxData, 1,
    1000);
HAL_SPI_TransmitReceive(&hspi5, &slaveTxData, &slaveRxData, 1,
    1000);

if ((slaveRxData % 3) == 0)
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);

if ((masterRxData % 5) == 0)
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);

HAL_Delay(2000);
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- In order to execute the code in Listing 8.10, create an empty project; configure the pins PE2, PE4, PE5, and PE6 as SPI4_SCK, SPI4_NSS, SPI4_MISO, and SPI4_MOSI, respectively.
- Likewise, the pins PF6, PF7, PF8, and PF9 should be set as SPI5_NSS, SPI5_SCK, SPI5_MISO, and SPI5_MOSI, respectively.
- Then, connect the pin PE2 to PF7, PE4 to PF6, PE5 to PF8, and PE6 to PF9 with jumper cables.
- Afterward, enable the full duplex master mode of SPI4 and the full duplex slave mode of SPI5, and enable the hardware NSS pins.
- Set the PCLK2 clock to 1.6 MHz as shown in Chap. 6. Then, set the prescaler values of SPI4 to 16 to get 100 kHz baud rate.
- Finally, set frame format to Motorola, data size to 8 bits, first bit to MSB first, clock polarity to low and clock phase to 1 edge, and disable the CRC calculation.
- As the project is compiled and run, the counter values will be transmitted from master to slave and slave to master, and LEDs will toggle accordingly.

Listing 8.11 Usage of the SPI module in non-blocking operation mode, the C code

```
/* USER CODE BEGIN PV */
uint8_t masterTxData = 0;
uint8_t slaveTxData = 0;
uint8_t masterRxData;
uint8_t slaveRxData;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
if (GPIO_Pin == GPIO_PIN_0)
{
masterTxData++;
slaveTxData++;

HAL_SPI_TransmitReceive_IT(&hspi4, &masterTxData, &masterRxData,
    1);
HAL_SPI_TransmitReceive_IT(&hspi5, &slaveTxData, &slaveRxData, 1)
    ;
__NOP();
}
}

void HAL_SPI_TxRxCpltCallback(SPI_HandleTypeDef *hspi)
{
if (hspi->Instance == SPI4)
{
if ((masterRxData % 5) == 0)
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
__NOP();
}
else if (hspi->Instance == SPI5)
{
if ((slaveRxData % 3) == 0)
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
__NOP();
}
}
/* USER CODE END 0 */
```

- The second example repeats the first example given in Listing 8.10 in non-blocking mode.
- When the transmission and reception is complete, the LEDs are toggled in callback functions.
- The pin PA0, in which the push button on the STM32F4 board is connected to, is configured in interrupt mode.
- It triggers the SPI transmission when pressed. In order to execute the code in Listing 8.11, first enable the SPI interrupts in the first example.
- Then, replace the “main.c” file with code given in Listing 8.11. As the project is compiled and run, the counter values will be transmitted from master to slave and slave to master, and LEDs will toggle when the onboard push button is pressed.

Usage via C++ Language

- We next repeat the first example given in the previous section now in C++ language. We provide the corresponding code in Listing 8.12.
- To execute the code, first make necessary hardware connections. Then, copy and paste the code to the “main.cpp” file in the active project and run it.
- This code performs the same operations as its C version.

Listing 8.12 Usage of the SPI module in blocking operation mode, the C++ code

```
#include "mbed.h"

#define WAIT_TIME_MS 2000

DigitalOut greenLED(LED1);
DigitalOut redLED(LED2);
SPI spiMaster(PE_6, PE_5, PE_2, PE_4);      // SPI4
SPISlave spiSlave(PF_9, PF_8, PF_7, PF_6); // SPI5

uint8_t masterTxData = 0;
uint8_t slaveTxData = 0;
uint8_t masterRxData;
uint8_t slaveRxData;

int main()
{
spiMaster.frequency(100000);
spiMaster.format(8, 0);
spiSlave.frequency(100000);
spiSlave.format(8, 0);

while (true)
{
masterTxData++;
slaveTxData++;

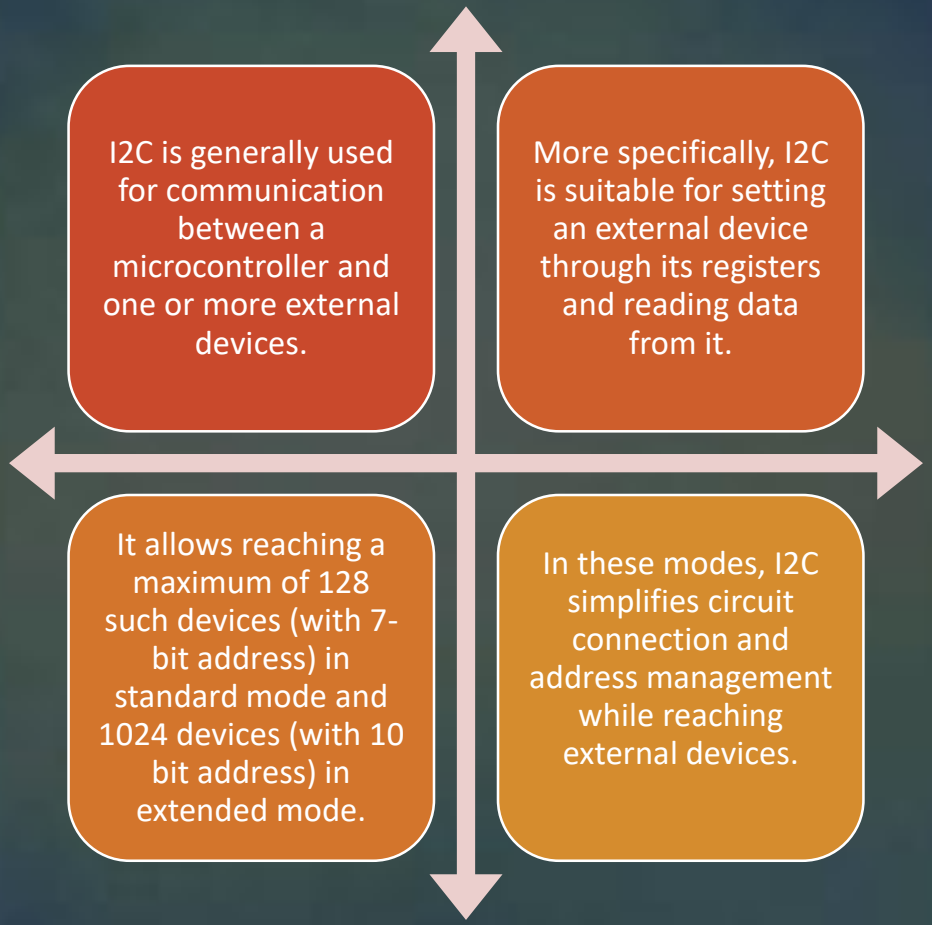
spiMaster.select();
spiMaster.write(masterTxData);
masterRxData = spiMaster.write(0x00);
if (spiSlave.receive())
{
slaveRxData = spiSlave.read();
spiSlave.reply(slaveTxData);
}
spiMaster.deselect();

if ((slaveRxData % 3) == 0)
greenLED = !greenLED;

if ((masterRxData % 5) == 0)
redLED = !redLED;

thread_sleep_for(WAIT_TIME_MS);
}
}
```

Inter-integrated Circuit
I2C



- I2C has a frame structure with start, address byte, ACK, data, and stop fields. We provide a generic frame in Fig. 8.15 to explain these concepts better.
- I2C uses NRZ for data encoding. I2C works in half duplex mode. I2C has both the master and slave modes.
- The clock in synchronous communication is generated by the master device. Moreover, the communication is always started by the master device.
- I2C has standard and fast data transfer modes. In the former mode, the maximum baud rate that can be reached is 100 kbits/s. In the latter mode, there are several sub options as fast-mode, fast-mode plus, and high-speed (HS) mode.
- Maximum baud rates that can be reached by these are 400 kbits/s, 1000 kbits/s, and 3.4 Mbits/s, respectively.

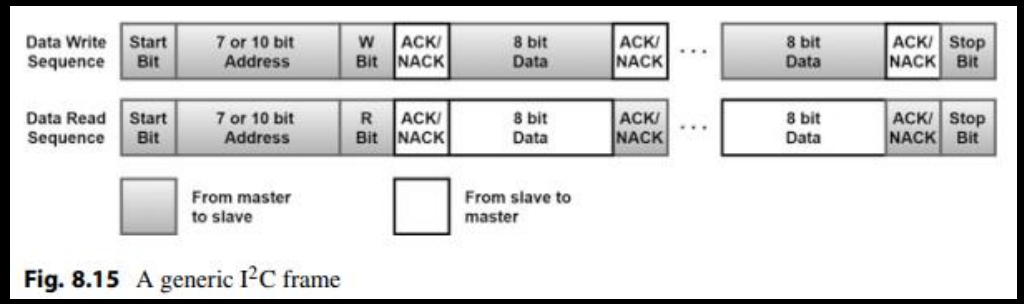


Fig. 8.15 A generic I^2C frame

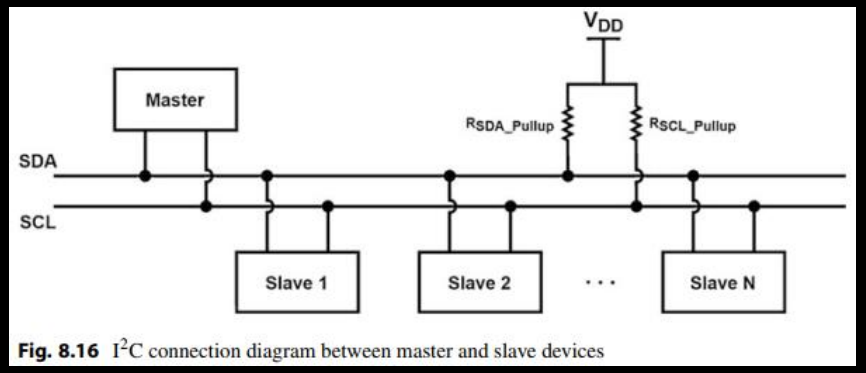


Fig. 8.16 I^2C connection diagram between master and slave devices

- In order to explain working principles of I2C, we should take a look at the connection diagram between the master and slave devices in Fig. 8.16.
- As can be seen in this figure, I2C has two pins in operation as clock (SCL) and data (SDA).
- Initially, the data line is in receive (input) mode for the slave device and transmit (output) mode for the master device.
- Then, the master device sends address of the slave device it needs to communicate with.
- Afterward, the master device can send or receive data to the slave.
- This is declared in the read/write bit sent along the address.
- The slave device knows what to do by checking this bit. The slave device sets its data pin accordingly, sends the acknowledge (ACK) signal, and sends data to the master if necessary.

I2C Modules in the STM32F4 Microcontroller

The STM32F4 microcontroller has three I2C modules as I2C1, I2C2, and I2C3. The first two modules are connected to the LCD and external RAM on the STM32F4 board. Therefore, they should be used with caution.

Each I2C module in the STM32F4 microcontroller can work in either master or slave mode. They can work as master transmitter, master receiver, slave transmitter, and slave receiver. Each module waits in slave mode by default.

Whenever a module needs to transmit data, it generates the clock signal and starts working as master. As data transmission ends, the module turns back to slave mode again.

This setup allows multi-master operation such that each connected module in the line can work as both master and slave device in different times. Each slave device has a programmable address composed of 7 or 10 bits.

Moreover, there may be two addresses for each device. In this setting, the module can work as if it is two different devices.

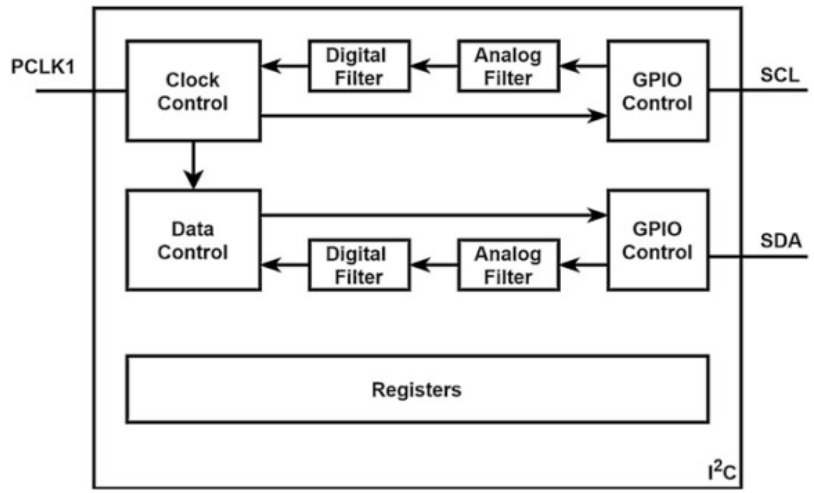


Fig. 8.17 Block diagram of an I^2C module in the STM32F4 microcontroller

- I2C modules use the APB1 peripheral clock (PCLK1). There are two speed modes as standard and fast with 100 kHz and 400 kHz clock frequency, respectively. In order to have the fast mode, the APB1 clock should be set to a value higher than 4 MHz.
- As can be seen in this figure, there are two pins in the module as SCL and SDA as explained before. These pins are set in open-drain mode. Besides, they are bidirectional. Hence, each pin can work either as output or input.
- To perform these operations, there is a specific GPIO control block. The clock control module is responsible for clock signal generation if the I2C module works as the master device. If the I2C module works as a slave device, then the clock control module works to sample the incoming (data) signal.
- The data control block is responsible for reading and writing data. The register block keeps all the information necessary to set the I2C module properties.

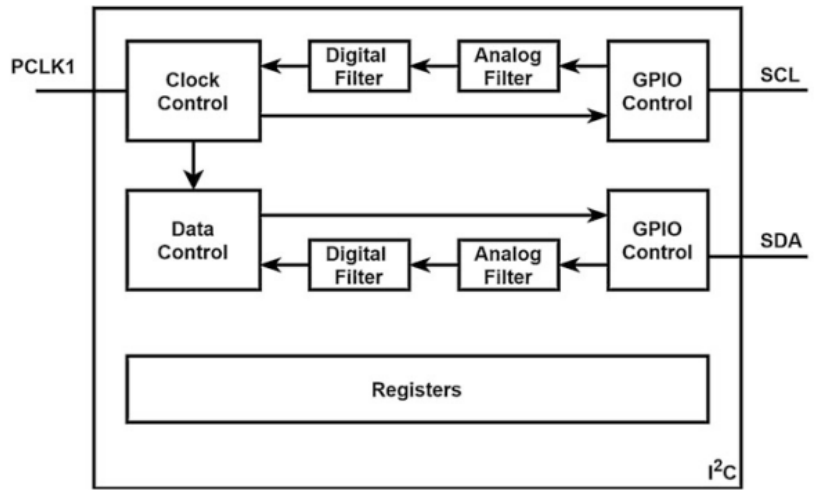
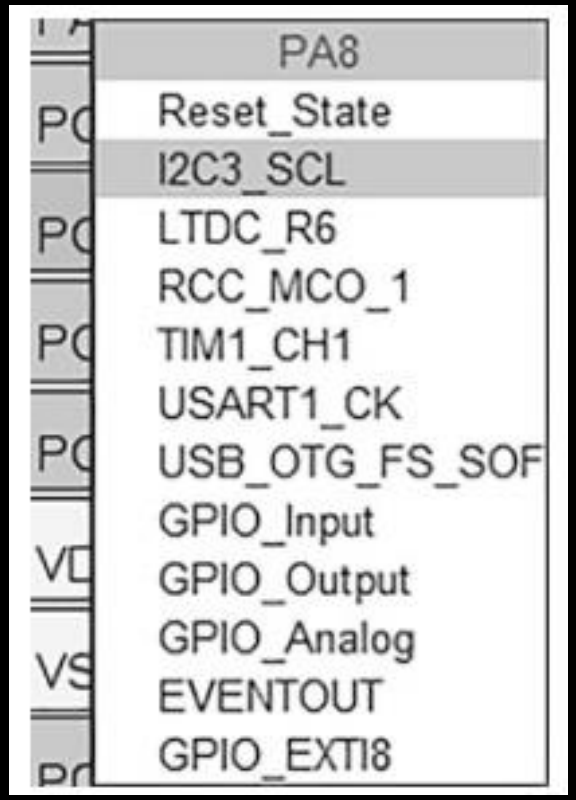


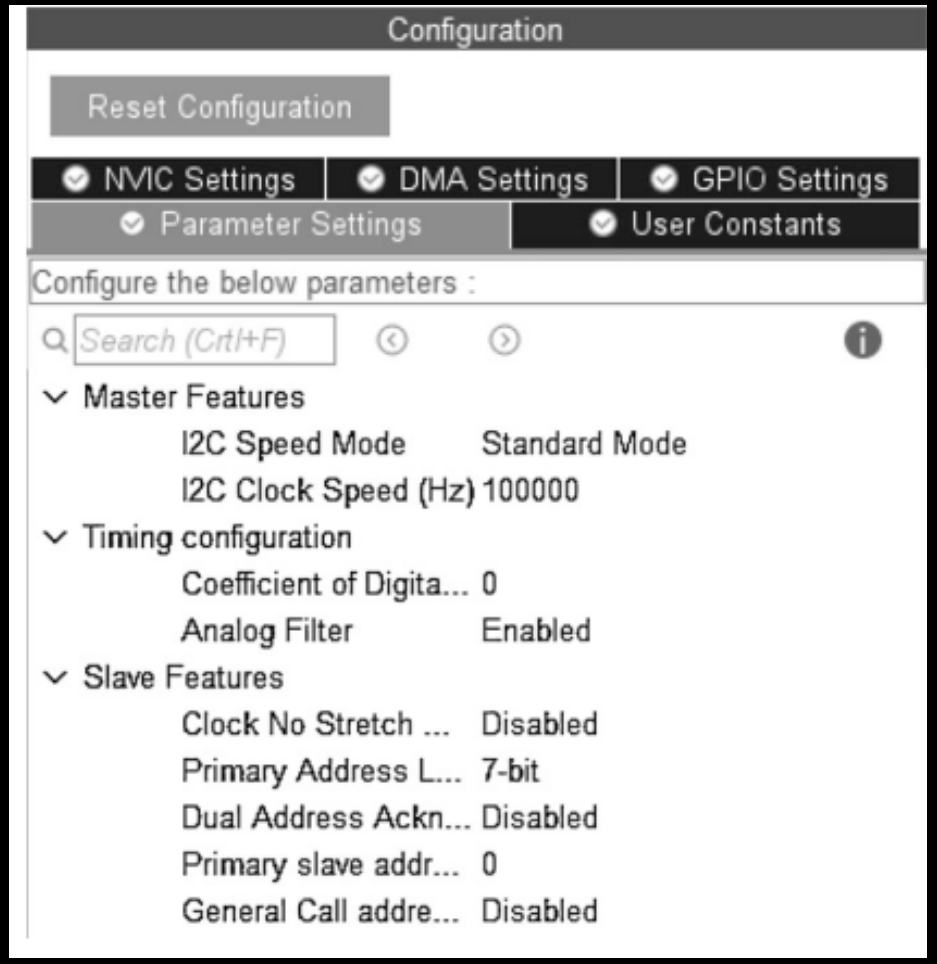
Fig. 8.17 Block diagram of an I^2C module in the STM32F4 microcontroller

- There is a maximum time limit to suppress spikes in data and clock signals I2C standard. These time limits are 50 ηs for standard and fast plus modes. It becomes 10 ηs for the high-speed (HS) standard. There are two filter blocks to suppress these spikes in the designated time in the STM32F4 microcontroller. These are labeled as analog and digital filters in Fig. 8.17. The analog filter has fixed parameters. Hence, it can only be enabled or disabled. On the other hand, the coefficients in the digital filter can be adjusted.

I2C Setup in the STM32F4 Microcontroller

- Then, configuration can be done as shown in Fig. 8.18. We will provide details of this operation next.
- In order to use the I2C module, SCL and SDA pins should be configured first. To do so, click on the GPIO pin with I2C capability in the STM32F4 microcontroller in the STM32CubeMX interface.
- Afterward, select the GPIO type as “I2Cx_SDA” or “I2Cx_SCLA” in the “Device Configuration Tool” window as in Fig. 8.18. To note here, x can be 1, 2, or 3 to denote I2C1, I2C2, or I2C3, respectively.





- After configuring the I2C mode, its settings can be configured from the opened “Parameter Settings” tab in the “Configuration” menu as in Fig. 8.20.
- Here, I2C speed mode can be selected as one of “Standard Mode” or “Fast Mode.” I2C clock speed can be set in Hz under “Master Features” section. To note here, “Fast Mode” is only available if the PCLK1 is set to work above 4 MHz.
- The primary address length can be selected as 7 or 10 bits in the “Slave Features” section. Associated to this value, primary slave address can be set to a value between 0 and 127 or 0 and 1023, respectively. To note here, the slave address will be shifted to left by 1 bit. For example, if you set the slave address as 80 (0x50 in hexadecimal form), then the actual address will be 160 (0xA0) for the read access and 161 (0xA1) for the write access.
- The analog and digital filters for SDA and SC lines are configured in the “Timing Configuration” section. Here, the analog filter can be enabled or disabled. Coefficient of the digital filter can be set a value between 0 and 15, where 0 disables the filter. To note here, the analog filter is enabled and digital filter is disabled by default.
- Similar to UART and SPI, I2C can operate in two operation modes as blocking (polling) or non-blocking (interrupt).
- If interrupt generation is not required, then this step can be skipped. Otherwise, the I2C interrupt should be enabled from the NVIC tab in the configuration menu as in Fig. 8.20.

Setup via C++ Language

- We can benefit from Mbed and available functions there to adjust the I2C module properties.
- To do so, we should first adjust SDA and SCK pins. This can be done by the classes I2C and I2CSlave for the master and slave modes, respectively.
- We should supply SDA and SCL pin names to these classes. Parameters of both class functions, such as frequency and slave address, can be configured after initialization.
- Moreover, the operation mode can be modified as blocking or non-blocking, and the callback function can be called whenever an interrupt occurs in non-blocking mode.

I2C Usage in the STM32F4 Microcontroller

- The first example, with the C code given in Listing 8.14, I2C2 and I2C3 modules are configured as I2C master and slave on the two boards, respectively.
- There are two counter values in the code. The first one will be sent to the slave from the master. Upon receiving this value by the slave, the green LED connected to pin PG13 of the STM32F4 board (working as slave) is toggled if mod 3 of the received counter value is zero.
- Afterward, the second counter will be sent to the master from slave. Upon receiving this value by master, the red LED connected to pin PG14 of the STM32F4 board (working as master) is toggled if mod 5 of the received counter value is zero. This operation will repeat every 2 s.
- In order to execute the code in Listing 8.14, create an empty project (for each board separately); configure the PB11 and PB10 pins as I2C2_SDA and I2C2_SCL, respectively. Also, set pins PC9 and PA8 as I2C3_SDA and I2C3_SCL, respectively.
- Afterward, enable the I2C mode of I2C2 and I2C3 modules. Set the I2C speed mode to standard, frequency to 100 kHz, and slave address to 80 (0x50).
- As the project is compiled and run, counter values will be transmitted from the master to slave and slave to master, and LEDs will toggle accordingly.

Listing 8.14 Usage of the I²C module in blocking transmit operation mode, the C code

```
/* USER CODE BEGIN PD */
#define MASTER
//#define SLAVE
/* USER CODE END PD */

/* USER CODE BEGIN PV */
#ifdef MASTER
uint8_t masterTxData = 0;
uint8_t masterRxData;
#endif

#ifdef SLAVE
uint8_t slaveTxData = 0;
uint8_t slaveRxData;
#endif
/* USER CODE END PV */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
#ifdef MASTER
masterTxData++;
HAL_I2C_Master_Transmit(&hi2c2, (uint16_t)160, &masterTxData, 1,
    10000);

HAL_I2C_Master_Receive(&hi2c2, (uint16_t)160, &masterRxData, 1,
    10000);
if ((masterRxData % 5) == 0)
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);

HAL_Delay(2000);
#endif

#ifdef SLAVE
slaveTxData++;
HAL_I2C_Slave_Receive(&hi2c3, &slaveRxData, 1, 10000);
if ((slaveRxData % 3) == 0)
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
HAL_I2C_Slave_Transmit(&hi2c3, &slaveTxData, 1, 10000);
#endif
}
/* USER CODE END WHILE */

/* USER CODE END 3 */
```

Usage via C++ Language

- We can benefit from the available functions under Mbed to use the I2C module via C++ language.
- Here, the functions of I2C and I2CSlave classes will be of help. The function frequency of both classes can be used to configure the speed of I2C module.
- The functions write and read of the I2C class can be used to transmit and receive data in blocking mode.
- Moreover, the functions start and stop of the I2C class can be used to send start and stop conditions to the I2C line.
- The functions write and read of the I2CSlave class can be used to transmit and receive data in blocking mode. The address function sets the slave address.

- We next modify the example in Listing 8.14 to work under Mbed in C++ language.
- The reader should follow the steps given there to form the hardware setup.
- Then, the C++ code given in Listing 8.16 should be used for the master and slave devices (STM32F4 boards) to execute the example.

Listing 8.16 Usage of the I^2C module in blocking operation mode, the C++ code

```
#include "mbed.h"

#define MASTER
// #define SLAVE

#define WAIT_TIME_MS 2000

#ifdef MASTER
DigitalOut redLED(LED2);
I2C i2c(PB_11, PB_10); // i2c2
 uint8_t masterTxData = 0;
 uint8_t masterRxData = 0;
 #endif

 #ifdef SLAVE
 DigitalOut greenLED(LED1);
 I2CSlave i2c(PC_9, PA_8); // i2c3

 uint8_t slaveTxData = 0;
 uint8_t slaveRxData = 0;
 #endif

 #define SLAVEADDRESS 80

 int main()
 {

 i2c.frequency(100000);

 #ifdef SLAVE
 i2c.address(SLAVEADDRESS);
 #endif

 while (true)
 {

 #ifdef MASTER
 masterTxData++;
 i2c.write(SLAVEADDRESS, (const char *)(&masterTxData), 1, false);
 i2c.read(SLAVEADDRESS, (char *)(&masterRxData), 1);

 if ((masterRxData % 5) == 0)
 redLED = !redLED;

 thread_sleep_for(WAIT_TIME_MS);
 #endif

 #ifdef SLAVE
 if (i2c.receive() == i2c.NoData)
 {
 slaveRxData = i2c.read();
 if ((slaveRxData % 3) == 0)
 greenLED = !greenLED;

 slaveTxData++;
 i2c.write(slaveTxData);
 }
 #endif
 }
 }
```

Controller Area Network
CAN

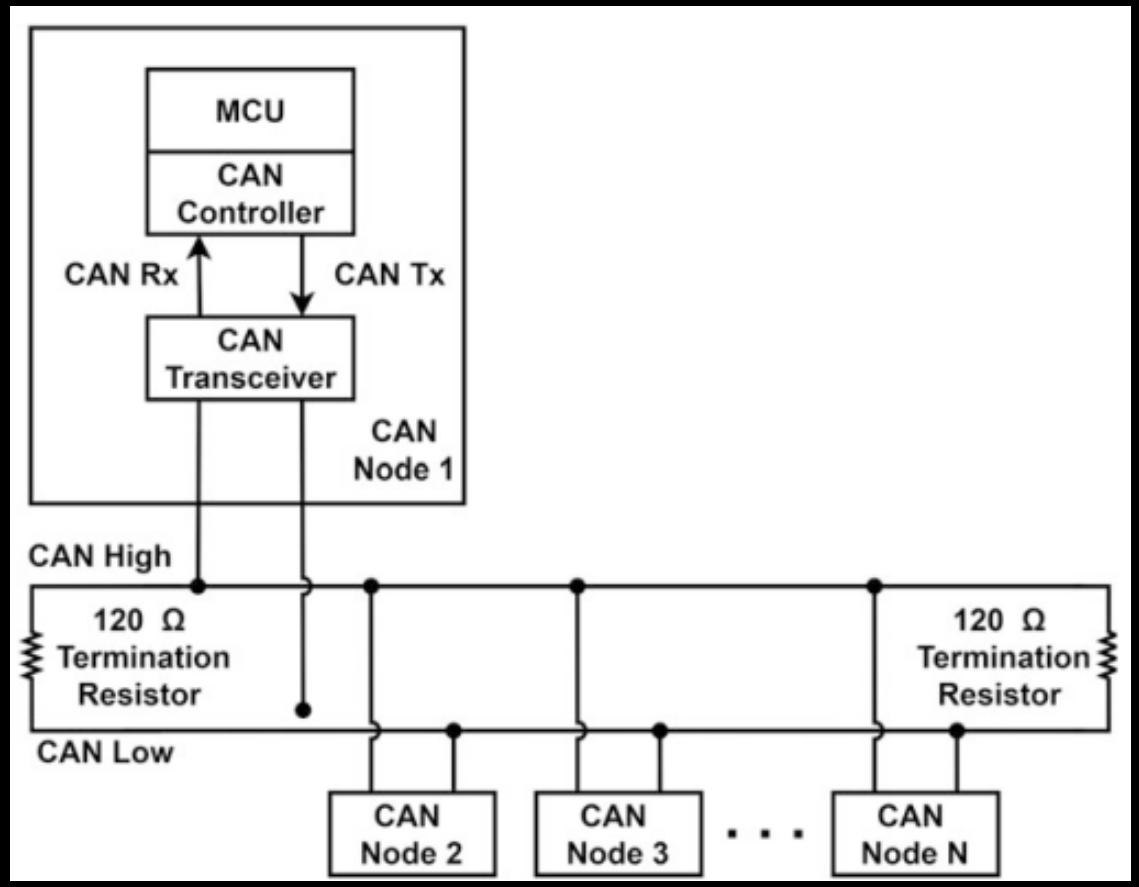
- CAN is the serial communication type extensively used in automotive industry. It has low error rate and does not need a main controller for operation. In CAN communication, there can be several devices connected via two cables (each having 120 terminal resistance). This line is called CAN bus.
- CAN bus forms the lowest two layers of the OSI (open systems interconnection) model as data link and physical layer. The data link layer is available in the microcontroller as the CAN module. The physical layer should be formed by an external transceiver which converts data to electrical CAN signal. Higher layers of OSI should be formed by the user in code.
- We can explain working principles of CAN (and CAN bus) based on the definitions in Sect. 8.1. CAN has five frames called data, extended data, remote, error, and overload. Among these, data frame is the most important one.

- Data frame is formed by start of frame (SOF) bit, arbitration field, control field, data field, CRC field, ACK, and EOF. SOF is always dominant (logic level 0).
- Arbitration field is composed of ID and remote transmission request (RTR) bit. ID is the 11 bit identification number of the submitted message. RTR indicates that the frame does not contain data and it will be sent as remote frame.
- Control field is composed of IDE, r0, and DLC.
    - Here, IDE is the identifier extension bit. If this bit has value logic level 0, 11 bit ID is used. Otherwise, 29 bit ID is used. This indicates the extended data frame. r0 is the reserved bit set to logic level 0.
    - DLC stands for the data length code. It is 4 bits and keeps size of the submitted data in bytes.
- Data field keeps the data to be submitted. It has length N×8 bits. N should be between 0 and 8. Hence, data length can be between 0 and 64 bits. If data length is zero, this indicates we have a remote frame. CRC stands for the cyclic redundancy check.
- The CRC field is composed of 16 bits of which 15 are for CRC and a logic level 1 bit as suffix. ACK is the acknowledge value composed of two bits. Each node sends back the ACK value to confirm data has been received. This is the case even if the node will not use data in the received frame. The ACK value will be 01 if there is no error in data reception. The ACK value will be 11 if there is error in data reception. Finally, EOF stands for the end of frame value which is composed of seven bits with logic level 1 value.

- Data encoding used in CAN is NRZ with one difference. Since 0 V corresponds to logic level 1 and a voltage value higher than 1.5 V represents logic level 0 in CAN, logic levels are inverted.
- However, the reader should not confuse this with NRZi data encoding. Instead, we will still call data encoding in CAN as NRZ with the mentioned logic level representations.
- There are two separate usages of half and full duplex modes in CAN. Data transfer between the microcontroller and transceiver is in full duplex mode. This is because there is two-sided data transfer in the CAN Rx and Tx pins. On the other hand, the CAN bus itself uses half duplex mode in communication since there is one-sided data transfer in the line.
- The master slave option is not applicable to CAN bus. Instead, there are specific protocols called CSMA/CD for this purpose. CSMA stands for carrier-sense multiple access. CD stands for collision detection.

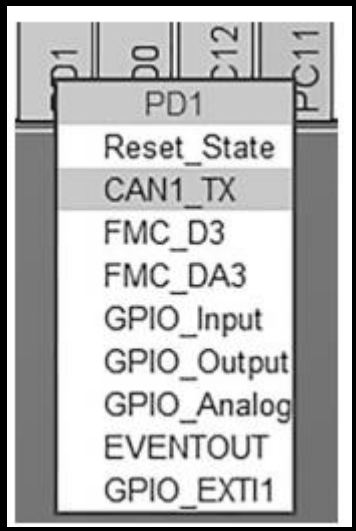
- The CAN bus does not carry a common clock signal. Hence, it is asynchronous. The bus speed is set to a constant value during design phase. The automotive environment has heavy noise. Moreover, data line lengths between nodes can reach 100 meters or more. Therefore, it is important for the nodes to synchronize the baud rate in communication. The first synchronization is done by the start bit at the beginning of data. This is called hard synchronization. If a synchronization shift is detected in the following operations, then synchronization is done again.
- Output voltage from the STM32F4 microcontroller is not compatible with the standard CAN bus voltage levels. Hence, there should be a transceiver connected to the microcontroller for CAN bus. In other words, the transceiver converts the signals at the CAN Tx and Rx pins of the microcontroller to CAN bus voltage levels.
- The CAN bus has a differential line. Let's call the lines in this setup as CAN high and CAN low. The difference of the two signals (CAN high - CAN low) will give us the actual CAN signal. Let's call this difference as CANDiff. In representing logic level 1, CAN high and low become 2.5 V. Hence, CANDiff becomes 0 V, whereas, in representing logic level 0, it is sufficient to have CANDiff to have a voltage value higher than 1.5 V. In the CAN bus standard, logic levels 0 and 1 are called dominant and recessive, respectively.

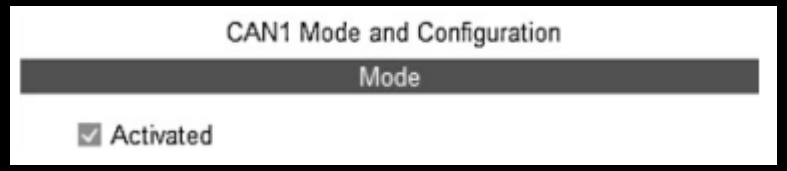
- As can be seen in this figure, the CAN bus is composed of two wires as CAN high and low. These wires are connected by terminal resistances with 120 value. There may be more than one node connected to the bus. Each node is composed of one CAN controller, CAN transceiver, and software on the microcontroller for CAN communication.
- CAN transceiver is connected to the CAN bus by CAN high and low pins. As a reminder, these pins form a differential pair. With this setting, the CAN transceiver converts data coming from CAN controller to CAN differential voltage.
- CAN controller and CAN transceiver are connected by CAN Rx and Tx pins. The CAN Rx pin transmits data coming from transceiver to data controller. The CAN Tx pin transmits data from data controller to transceiver.



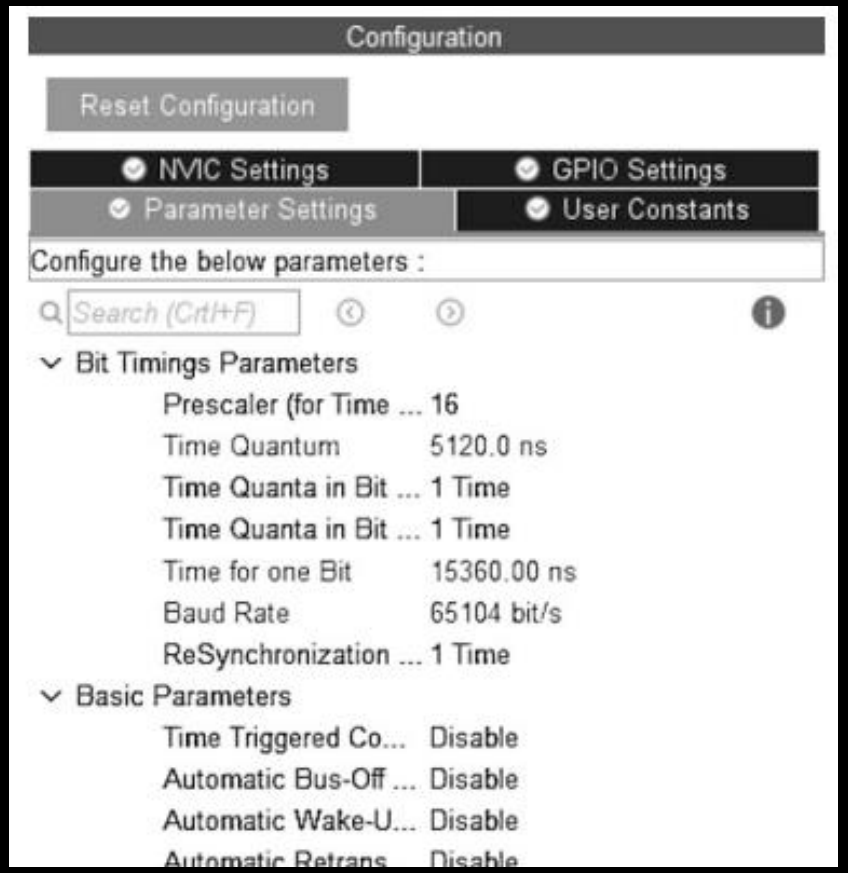
CAN Modules in the STM32F4 Microcontroller

- In order to use the CAN module, Tx and Rx pins should be configured first. To do so, click on the GPIO pin with CAN capability on the STM32F4 microcontroller in STM32CubeMX interface. Afterward, select the GPIO type as “CANx_TX” or “CANx_RX” in the “Device Configuration Tool” window as in Fig. 8.23. To note here, x can be 1 or 2 to denote CAN1 or CAN2, respectively.
- Once CAN pins are selected and configured, click the set CAN module under the “Connectivity” section from the left STM32CubeMX menu. Then, mode configuration can be done in this section as in Fig. 8.24. Here, the CAN module can be activated by clicking the “Activated” checkbox.

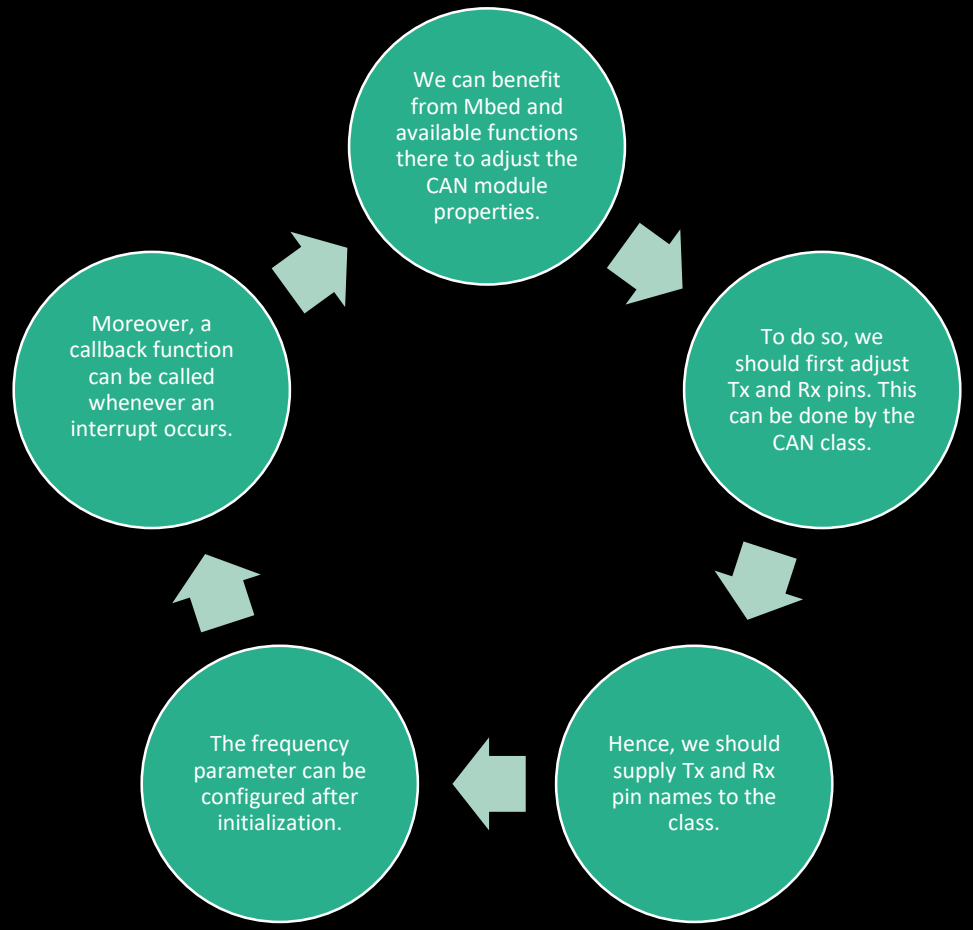




- After activating the CAN module, its parameters can be set from the opened “Parameter Settings” tab in the “Configuration” menu as in Fig. 8.25.
- Here, baud rate of the CAN module can be configured using “Prescaler (for Time Quantum),” “Time Quanta in Bit Segment 1,” and “Time Quanta in Bit Segment 2” options under “Bit Timing Parameters.” We will use the “Automatic Retransmission” option under “Basic Parameters” and “Operating Mode” to select the CAN mode under “Advanced Parameters.”
- The rest of the options can be left in their default settings. The “Automatic Retransmission” can be selected as “Enabled” or “Disabled.” This is to set or reset resending the data packet if it fails in the first attempt. The “Operating Mode” can be selected as “Normal,” “Loopback,” “Silent,” or “Loopback combined with Silent.”



Setup via C++ Language



- We next provide an example on the usage of the CAN module. Here, CAN1 module is configured in loopback mode. Hence, the code block for message sending and receiving parts can be executed on the same microcontroller.
- As a result, the code can be easily debugged. There is a counter value in the code that will be transmitted and received every 2 s.
- If mod 3 of the received counter value is zero, then the green LED connected to pin PG13 of the STM32F4 board is toggled. If mod 5 of the received counter value is zero, then the red LED connected to pin PG14 of the STM32F4 board is toggled.

Listing 8.19 Usage of the CAN module, the C code

```
/* USER CODE BEGIN PV */
CAN_FilterTypeDef sFilterConfig;
CAN_TxHeaderTypeDef sTxHeader;
CAN_RxHeaderTypeDef sRxHeader;
uint32_t TxMailbox;

uint8_t canTxData[8] = {0};
uint8_t canRxData[8];
/* USER CODE END PV */

/* USER CODE BEGIN 2 */
sFilterConfig.FilterBank = 0;
sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;
sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;
sFilterConfig.FilterIdHigh = 0x12 << 5;
sFilterConfig.FilterIdLow = 0x0000;
sFilterConfig.FilterMaskIdHigh = 0x0000;
sFilterConfig.FilterMaskIdLow = 0x0000;
sFilterConfig.FilterFIFOAssignment = CAN_RX_FIFO0;
sFilterConfig.FilterActivation = ENABLE;
sFilterConfig.SlaveStartFilterBank = 14;

HAL_CAN_ConfigFilter(&hcan1, &sFilterConfig);
HAL_CAN_Start(&hcan1);

sTxHeader.StdId = 0x12;
sTxHeader.RTR = CAN_RTR_DATA;
sTxHeader.IDE = CAN_ID_STD;
sTxHeader.DLC = 2;
sTxHeader.TransmitGlobalTime = DISABLE;
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
canTxData[0]++;

HAL_CAN_AddTxMessage(&hcan1, &sTxHeader, canTxData, &TxMailbox);
while (HAL_CAN_IsTxMessagePending(&hcan1, TxMailbox));
HAL_CAN_GetRxMessage(&hcan1, CAN_RX_FIFO0, &sRxHeader, canRxData)
    ;

if ((canRxData[0] % 3) == 0)
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);

if ((canRxData[0] % 5) == 0)
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);

HAL_Delay(2000);
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

C++

Listing 8.20 Usage of the CAN module, the C++ code

```
#include "mbed.h"

#define WAIT_TIME_MS 2000
#define filterHandle 0

DigitalOut greenLED(LED1);
DigitalOut redLED(LED2);

CAN can(PA_11, PA_12);

uint8_t canTxData = 0;

int main()
{
CANMessage canRxData;
CANMessage TxMsg(0x12, &canTxData, 1, CANData, CANStandard);

can.mode(CAN::LocalTest);
can.filter(0x12, 0xFF, CANStandard, filterHandle);

while (true)
{
if (can.write(TxMsg))
{
canTxData++;
}

if (can.read(canRxData, filterHandle))
{
if ((canRxData.data[0] % 3) == 0)
greenLED = !greenLED;
if ((canRxData.data[0] % 5) == 0)
redLED = !redLED;
}
thread_sleep_for(WAIT_TIME_MS);
}
}
```

Universal Serial Bus
USB

- USB is a standard aiming to connect the PC, cell phones, tablets, and peripheral devices such as mouse, keyboard, and webcam. Besides, USB also sets the standards for the cable, connector, voltage level, signal type, and protocols used in operation. USB has three major versions as USB1.x, USB2.x, and USB3.x. Each major version also has its subversions.
- Data in USB is represented in layers. The highest layer is the frame. There are transactions within a frame. These consist of packets. Each packet is composed of fields.
- A device need not to be a peripheral all the time in USB. There is a specific property developed for it called USB on the go (OTG). A good example for this operation is the cell phone. If the cell phone is connected to a PC, the PC acts as host and cell phone acts as peripheral. If a USB memory is connected to the cell phone, then the cell phone changes its mode and acts as host, and USB memory acts as peripheral. This is achieved since the cell phone supports the USB OTG standard.

- Data transfer speeds in USB are set by standards and these cannot be changed. Besides, there are modes in USB versions (specific to USB 2.x and USB 1.x) as low speed (LS), full speed (FS), and high speed (HS). Based on these, data transfer speeds in terms of bit rates are tabulated in Table 8.1. Since the STM32F4 microcontroller supports the USB2.0 version, we will just focus on it.
- Output voltage from the microcontroller (such as STM32F4) may not be compatible with the standard USB line voltage levels. Hence, there should be a transceiver (specifically called PHY) connected to the microcontroller. In other words, the transceiver converts signals at the microcontroller output to actual USB line voltage levels. As a side note, the STM32F4 microcontroller has an embedded PHY within itself.
- USB uses a differential line with line names D+ and D-. The STM32F4 microcontroller reference manuals use the DP and DM naming for these, respectively. Data encoding used in operation is NRZI. Besides encoding, D+ and D- line voltage values are used to define states. The USB line works in half duplex mode.

Mode	Acronym	Bit rate (Mbits/s)	USB version
Low speed	LS	1.5	USB 1.0
Full speed	FS	12	USB 1.1
High speed	HS	480	USB 2.0

- We can explain working principles of USB based on its layered structure as in Fig. 8.26. USB frame is started at every millisecond in USB LS and FS. In USB HS, this timing becomes 125 µs.
- A peripheral device (such as mouse) cannot send data by itself. Instead, it can send data as a response to a frame started by the host.
- As can be seen in Fig. 8.26, each frame consists of one start of frame (SOF) token and more than one transaction. Each transaction is composed of three packets as token, data, and handshake.
- The data packet is optional. Each packet is composed of different fields as SYNC, PID, ADDR, ENDP, PAYLOAD, CRC, and EOP. The SYNC field is 8 and 32 bits for USB LS and USB HS, respectively. This field is responsible for clock synchronization between the host and peripheral devices. The PID stands for packet ID. This field is 8 bits and defines type of the packet. ADDR is the 7-bit address field. Through it, the receiver is set if the hub option is used in communication. ENDP is the 4-bit endpoint address. There are virtual endpoints in each peripheral device defining the function of the USB device. PAYLOAD is the transmitted data which can be between 0 and 1023 bytes. Token packets have 5 bit CRC. Data packets have 16 bit CRC. EOP represents the end of packet.

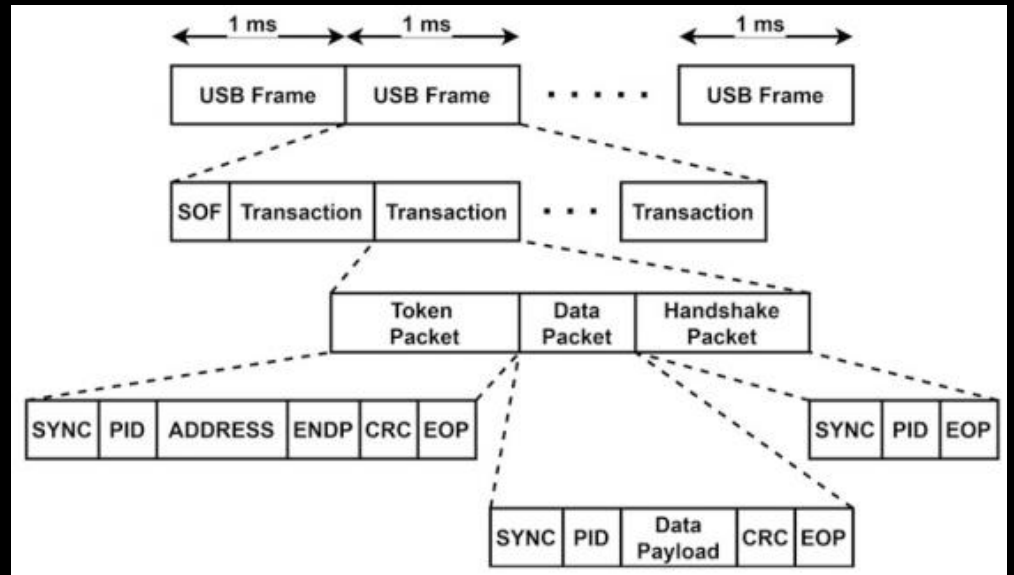
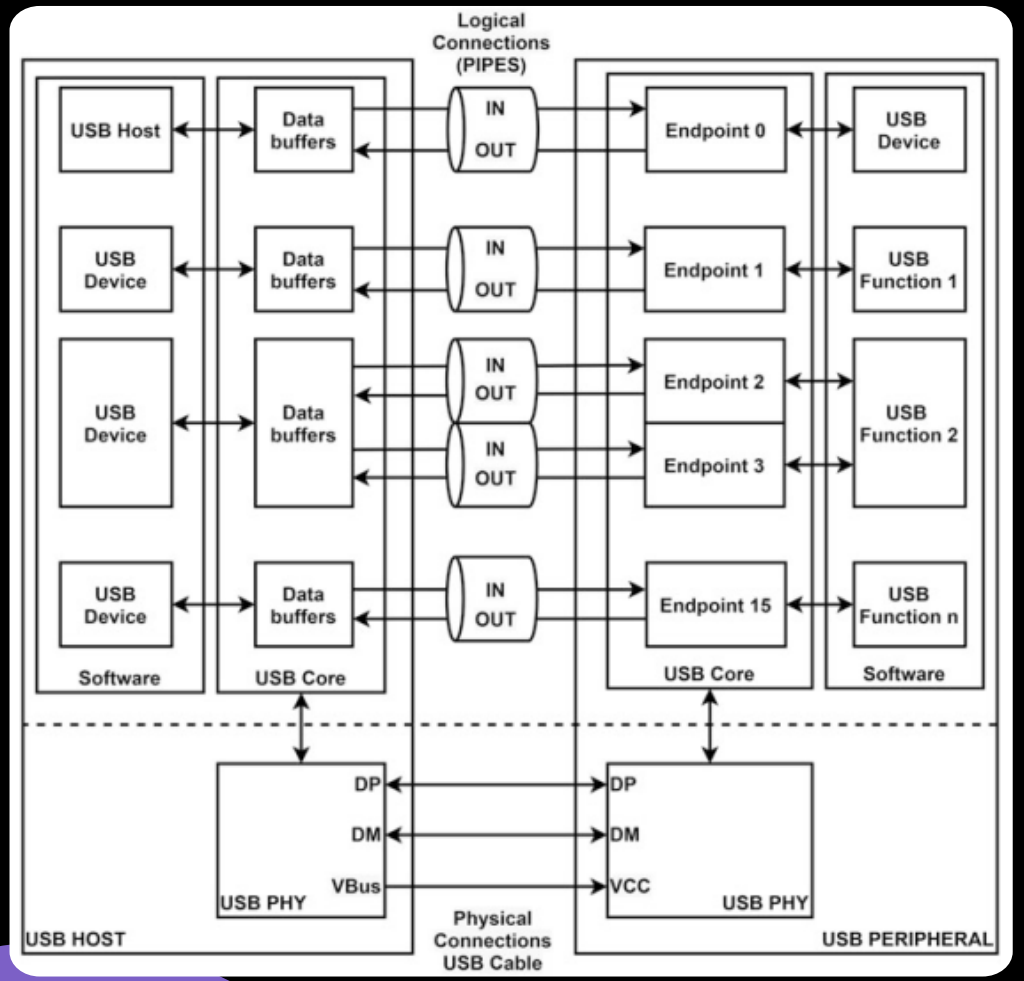


Fig. 8.26 USB frame



- We next provide the general USB connection diagram between a host and peripheral device in Fig. 8.27. As can be seen in this figure, the USB line is composed of two data lines and one VBUS line. Data lines can be used in both differential and single-ended form.
- The host and peripheral blocks are composed of one USB core (controller) and USB PHY. The latter is used to adjust voltage level between the USB core and USB line. Each peripheral has at most 16 endpoints (16 in, 16 out). Endpoints can be grouped within themselves. These groups are called transfers.

USB Modules in the STM32F4 Microcontroller

- The STM32F4 microcontroller has two USB modules. One of them is the USB FS and the other is USB HS. Both modules are compatible with the USB 2.0 standard. They also support the USB OTG property.
- As a reminder, USB FS supports transmission speed of 12 Mbits/s. USB HS supports transmission speed of 480 Mbits/s. Moreover, both modules support low-speed (LS) 1.5 Mbits/s transmission. PHY is also available in the STM32F4 microcontroller.
- The USB FS and USB HS modules have their own FIFO RAMs. These are used to receive and transmit data. The USB FS module FIFO RAM has size of 1.25 kB. The USB HS module FIFO RAM has size of 4 kB. Each FIFO RAM can hold multiple packets.

- We provide block diagram of the USB FS and USB HS modules of the STM32F4 microcontroller in Fig. 8.28.
- In this figure, DP and DM pins are for data transfer. VBUS is for 5 V power supply.
- ID pin is used to check whether the device is host or peripheral in the OTG operation. PHY converts the USB line electrical signal to standard logic levels and transfers them to the USB core module.
- As all frame operations are done in the USB core module, data is transferred to the related FIFO RAM. USB FS and USB HS modules work at 48 MHz clock speed.

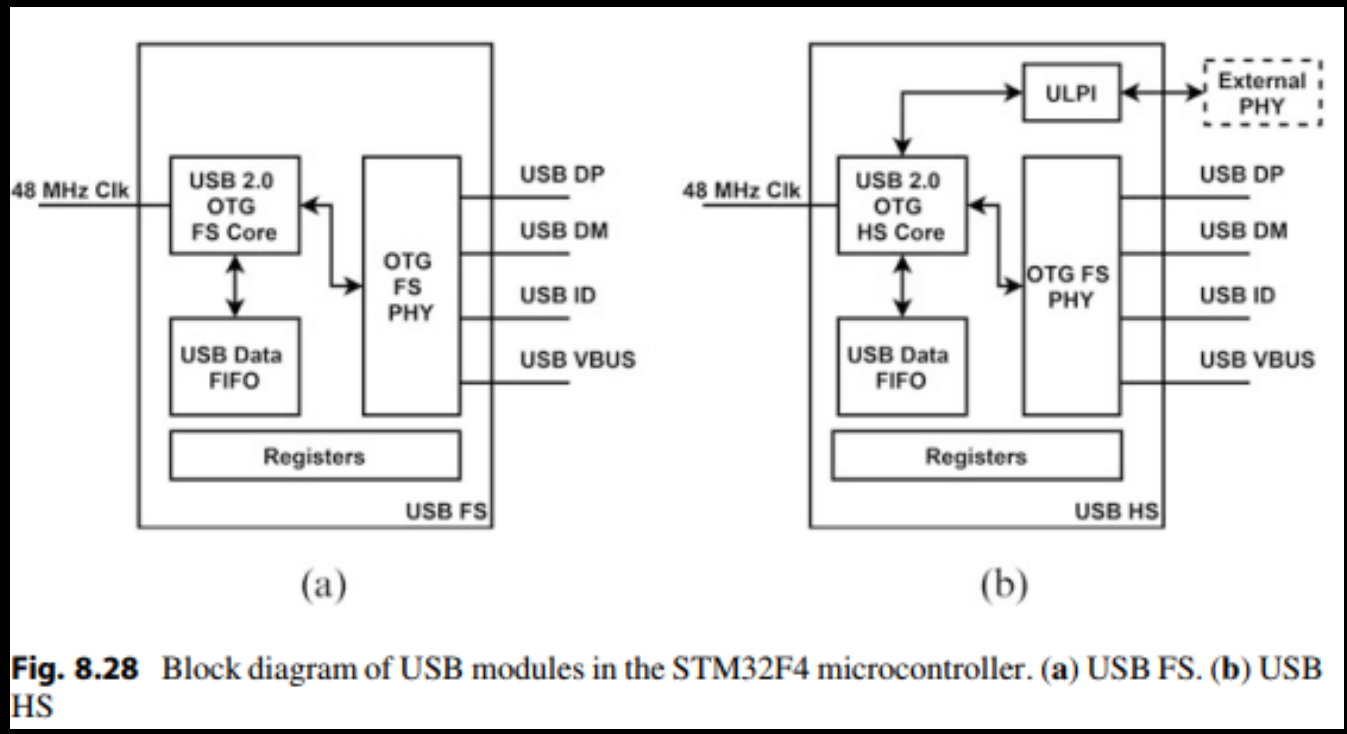


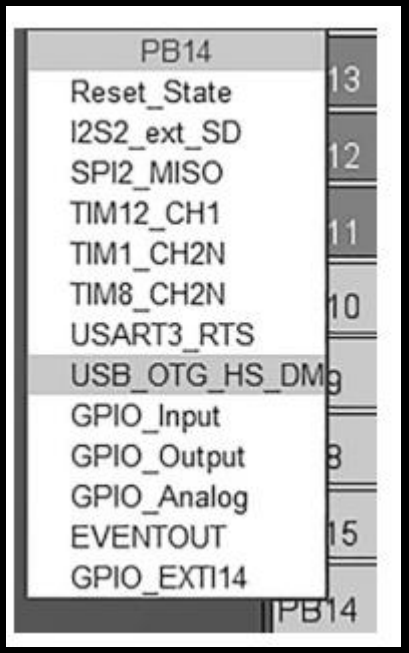
Fig. 8.28 Block diagram of USB modules in the STM32F4 microcontroller. (**a**) USB FS. (**b**) USB HS

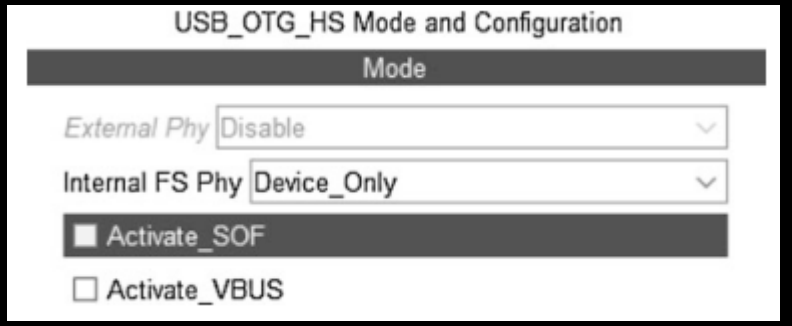
- The USB FS and USB HS modules in the STM32F4 microcontroller have two modes as host and peripheral.
- In the host mode, 5 V voltage should be powered to the connected peripheral device. This is called VBUS. The host is responsible for this operation done by the external charge pump circuit.
- In the host mode, USB FS and HS support 8 and 12 connection channels. This means eight different functions can be supported or data can be received or sent in USB FS module. The number of functions becomes 12 in the USB HS module.
- In the peripheral mode, USB FS module supports one bidirectional (three in, three out) endpoint. This means the module can support one microphone, one headphone, and their control keys at the same time. USB HS module supports one bidirectional (five in, five out) endpoint.

USB Setup in the STM32F4 Microcontroller

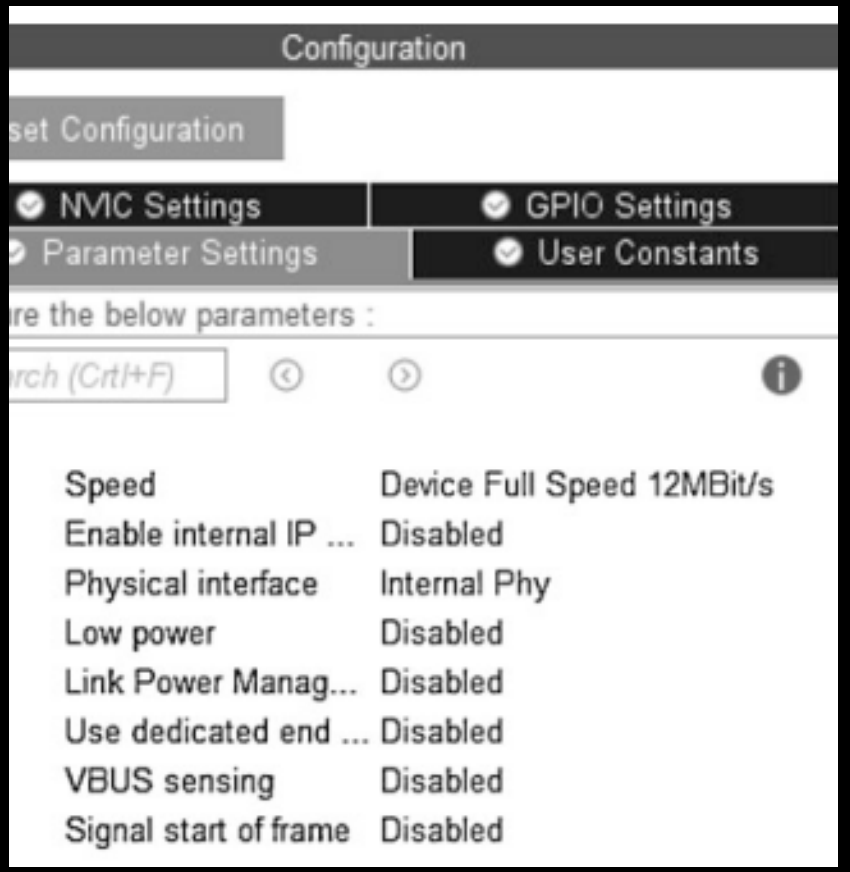
- In order to use the USB HS or USB FS module, DM and DP pins should be configured first. To do so, click on the GPIO pin with USB capability of the STM32F4 microcontroller in the STM32CubeMX interface.
- Afterward, select the GPIO type as “USB_OTG_FS_DM,” “USB_OTG_FS_DP,” “USB_OTG_HS_DM,” or “USB_OTG_HS_DP,” in the “Device Configuration Tool” window.

- Once the DP and DM pins are selected, click the appropriate option (“USB_OTG_FS” or “USB_OTG_HS”) under the “Connectivity” section from the left STM32CubeMX menu. Then, mode configuration can be done in this section as shown in Fig. 8.30. Here, select the mode as “Device only.” The other modes can be “Host only” or “OTG.”



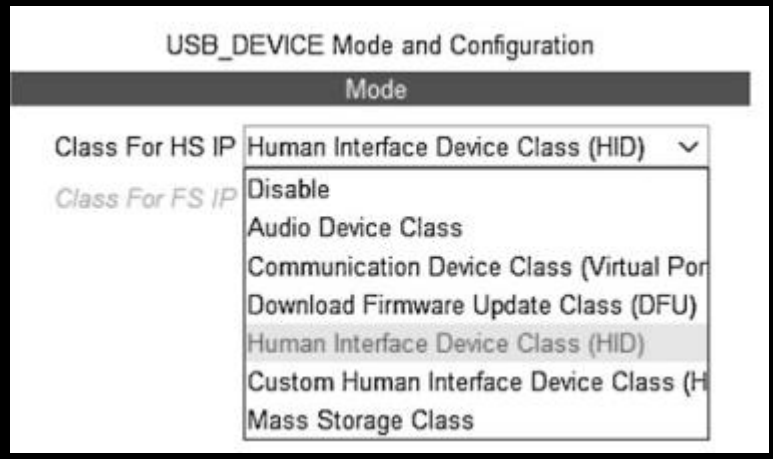


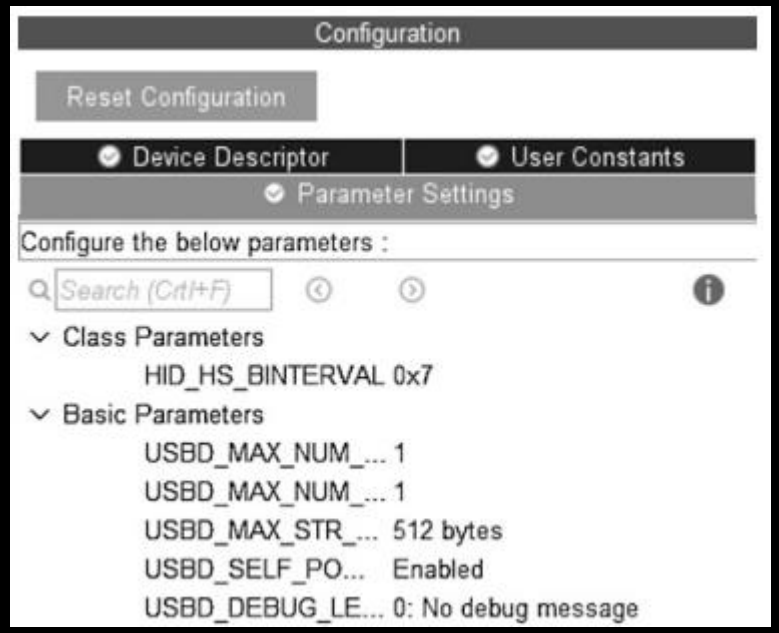
- After configuring the USB module in device mode, its parameters can be set from the opened “Parameter Settings” tab in the “Configuration” menu as in Fig. 8.31. At this stage, we will use the default settings which will be sufficient for us.
- We should also set the clock properties for the USB to work. To do so, the “48MHz clocks” clock output should be active. This clock is fed by the main PLL. Hence, the HSE oscillator should also be activated. To do so, select the “Crystal/Ceramic Resonator” as high-speed oscillator (HSE) under the STM32CubeMX RCC window. Afterward, the main PLL input should be selected as HSE in the clock configuration window. Related to all these settings, we should also set the PCLK to 168 MHz.



- To use the USB module, we can benefit from ST USB middleware. Therefore, click on to “USB_DEVICE” under the “Middleware” section from the left STM32CubeMX menu. Then, mode configuration of USB middleware can be done in this section as shown in Fig. 8.32. Here, select “Human Interface Device Class (HID).”

- After configuring the USB middleware in “Human Interface Device Class (HID),” its settings can be configured from the opened “Parameter Settings” tab in the “Configuration” menu as in Fig. 8.33. At this stage, we will use the default settings which will be sufficient for us.





Usage via C Language

- The first example, with the code given in Listing 8.22. Here, the USB HS module will be configured as USB device. To do so, the USB_Device middleware will be used to implement a USB mouse. The onboard button connected to pin PA0 will be configured in interrupt mode. It will trigger the transmission when pressed. Also, digital value of this button will be sent to host PC as left mouse button click. Hence, the user can use the onboard button as the left mouse button.
- In order to run the code in Listing 8.22, create an empty project; configure the pins PB14 and PB15 as USB_OTG_HS_DM and USB_OTG_HS_DP, respectively. Set the internal USB FS PHY of USB_OTG_HS as “Device Only.” Then, set the “Human Interface Device Class (HID)” of class for USB HS of USB_DEVICE middleware. Enable the crystal/ceramic resonator for HSE and set 48 MHz clock output. Configure the PA0 pin in external interrupt mode with falling/rising trigger detection. As the project is compiled and run, the onboard button can be used as left mouse button. The user can test the code by moving the mouse over a file on the PC desktop and then pressing the onboard button.

Listing 8.22 Usage of the USB module, first example, the C code

```
/* USER CODE BEGIN Includes */
#include "usbd_hid.h"
/* USER CODE END Includes */

/* USER CODE BEGIN PV */
// HID Mouse
struct mouseHID_t
{
uint8_t buttons;
int8_t x;
int8_t y;
int8_t wheel;
};
struct mouseHID_t mouseHID = {0, 0, 0, 0};

extern USBD_HandleTypeDef hUsbDeviceHS;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
if (GPIO_Pin == GPIO_PIN_0)
{
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET)
mouseHID.buttons = 1;
else
mouseHID.buttons = 0;
USBD_HID_SendReport(&hUsbDeviceHS, (uint8_t *)&mouseHID, (
    uint16_t)sizeof(struct mouseHID_t));
__NOP();
}
}
/* USER CODE END 0 */
```

SD Bus Interface

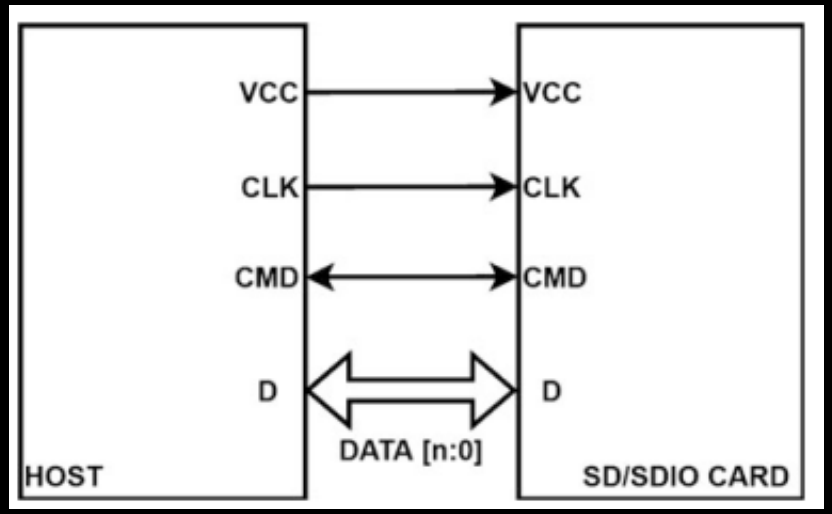
- SD bus is used to communicate with the SD card or other devices attached to the SD card slot. SD bus interface is the communication type used for this purpose. Secure digital input and output (SDIO) is the peripheral unit in the STM32F4 microcontroller performing SD bus interface operations.
- Recently SD cards have been used in embedded systems. All SD card systems support both SPI and SD bus interface. The connected device decides on which one to use.

- We can explain working principles of the SD bus interface based on the definitions in Sect. 8.1 as follows.
- Data types and encoding options are similar to SPI. Hence, NRZ is used in operation.
- Voltage level used in operation is 3.3 V. Some cards can switch to 1.8 V after start-up. This is done to save power.
- SD bus interface can use one cable for serial communication. It can also communicate in parallel form with four or eight cables. Based on these, the communication is called 1-bit, 4-bit, or 8-bit wide. Normally, all SD cards support 1-bit-wide communication. If there is extra support for 4- or 8-bit-wide communication, then the SD card can switch to these modes.

Card type	Max bit Clock (MHz)	Max transfer rate (Mbits/s) 1-bit wide	4-bit wide	8-bit wide
MMC	20	20	–	–
RS-MMC	20	20	–	–
MMCplus	52	52	208	832
MMCmobile	52	52	208	832
SecureMMC	20	20	–	–
SD	208	208	832	–
MiniSD	208	208	832	–
MicroSD	208	208	832	–
SDIO	50	50	200	–

- SD bus interface has two different pin groups as command and data.
- The master device sends commands, such as read/write, via one (command) pin to the slave device.
- There are one, four, and eight pins for 1-, 4-, and 8-bit-wide data communication, respectively.
- The master device only sends data to be read or written to. Therefore, command and data pins work in half duplex mode.
- SD bus interface supports master and slave communication.
- However, they are called host and card, respectively.
- Communication speed in SD bus interface depends on the used SD card and number of data cables. We tabulated them in Table 8.2. As can be seen in this table, the maximum transfer rate can be 832 Mbits/s.

- Frames in SD bus interface are constructed in two different forms. The first one handles the commands sent through its pin. The second one is the data sent through data pin or pins.
- The host device sends the command frame with 48 bits. Fields forming this frame are start bit, transmission bit, command index, argument, CRC, and end bit. The start bit is always set to logic level 0. Likewise, the transmission bit is always set to logic level 1.
- The command index is composed of 6 bits keeping commands. Argument is composed of 32 bits keeping the argument. CRC is composed of 7 bits. The end bit is always set to logic level 1. The SD card responds to a command with frames having different structures.
- The host device sends the data frame in a structure as follows.
- Fields in the frame are start bit, data, CRC, and end.
- The start bit always has logic level 0 value.
- Data size is fixed to 512 bytes for some commands. For others, its size can change. However, the maximum size cannot exceed 512 bytes.
- CRC is composed of 16 bits. The end bit has always logic level 1 value.



- The connection diagram for the SD bus is as in Fig. 8.34. As can be seen in this figure, there is one clock line called CLK. The clock is provided by the host device. There are also one command (CMD) pin and n+1 data pins. Here, n can be 7, 3, or 0 based on the communication type. The host also supports power to the card

- Data transfer in the SD bus interface can be summarized as follows:
  - The host first sends the command to the SD card such as read, write, or erase. The SD card responds to this.
  - When the 1-bit-wide data line is used, data is transferred synchronously with the clock as in SPI communication.
  - When 4-bit-wide data line is used, there are two methods.
    - In the first method, most significant 4 bits of 1 byte data are sent in the first clock pulse; the remaining least significant 4 bits of data are sent in the second clock pulse. Hence, 1 byte of data is transferred after two clock pulses.
    - In the second method, each cable works independently. Hence, 4 bytes of data is taken. Most significant (seventh) bits of all 4 bytes are sent from four cables separately at the first clock pulse. The next sixth bits of all 4 bytes are sent from four cables separately at the second clock pulse. This operation repeats till the least significant (zeroth) bits of the 4 bytes are sent.
  - As a summary, 4 bytes of data is transmitted after eight clock pulses. The same method applies to 8-bit-wide data line. The only difference will be the number of used parallel lines, which is eight in this case. Hence, 8 bytes of data is transmitted after eight clock pulses.

- The STM32F4 microcontroller has one SDIO module. Through it, we can use SD cards supporting protocols MultiMediaCard system specification version 4.2, SD memory card specifications version 2.0, SD I/O card specification version 2.0, and CE-ATA digital protocol Rev1.1. The STM32F4 microcontroller supports 48 MHz clock speed for these operations. The module also supports 1-, 4-, and 8-bit-wide data transfer.



Inter-IC Sound
I2S

- Inter-IC sound (I2S) is extensively used for audio data transfer between ADC/DAC, codec integrated circuits (IC), and the microcontroller or DSP chips.
- We can explain working principles of I2S based on the definitions in Sect. 8.1. We should first mention that this protocol is derived from SPI. Hence, they have several common properties. The difference of I2S from SPI is that, in the former, the clock signal is always active. I2S uses NRZ for data encoding. I2S can work in full duplex or simplex mode. However, it is used in simplex mode most of the times. In I2S, there is master slave setup. Here, the module generating the clock signal is called master. The other IC becomes slave. Besides, there is also transmitter, receiver, and controller. The module transmitting data is called transmitter. The module receiving data is called the receiver. The controller decides on who will transmit data and who will receive it. This module need not to be available all the times. Based on these, there may be different combinations such as master transmitter, master receiver, or master controller.